

Dynamic RC-WTCTP Project Crashing

K13 Pemodelan Matematika

May 8, 2026

1 Pendahuluan

Dalam manajemen proyek modern, penyelesaian proyek tepat waktu dengan biaya minimal merupakan salah satu tantangan utama yang dihadapi oleh manajer proyek. Proyek sering kali menghadapi ketidakpastian, keterbatasan kapasitas sumber daya, dan tekanan tenggat waktu (**deadline**). Untuk mengatasi masalah ini, dikembangkan dua konsep optimisasi penting: **Resource-Constrained Project Scheduling Problem** (RCPSP) dan **Time-Cost Trade-off Problem** (TCTP).

RCPSP berfokus pada penjadwalan aktivitas proyek dengan memperhatikan batasan ketergantungan antar-aktivitas (**precedence constraints**) dan keterbatasan kapasitas sumber daya harian yang tersedia. Di sisi lain, TCTP mengasumsikan bahwa durasi pengerjaan suatu aktivitas dapat dipangkas (**project crashing**) dengan cara mengalokasikan sumber daya tambahan, yang konsekuensinya akan meningkatkan biaya proyek.

Secara umum, terdapat dua skenario **project crashing** yang dibahas dalam laporan ini:

1. **Skenario Baseline:** Menghadapi masalah **crashing** ketika biaya pemotongan durasi harian (**crash cost per day**) dan batas durasi minimum (**minimum duration**) untuk setiap aktivitas telah diketahui secara eksplisit dan bernilai konstan. Model ini dikembangkan menggunakan paradigma pemrograman batasan (**Constraint Programming**) melalui Google OR-Tools CP-SAT.
2. **Skenario Novel:** Menghadapi masalah **crashing** ketika biaya pemotongan durasi dan batas durasi minimum tidak diketahui secara langsung. Sebaliknya, manajer proyek hanya memiliki tuas kontrol berupa alokasi harian sumber daya manusia, yang dapat dipercepat melalui dua mekanisme: penambahan tenaga kerja (**overcrowding**) dan penambahan jam kerja lembur (**overtime**). Hubungan non-linier antara alokasi sumber daya tambahan dengan durasi aktivitas dimodelkan menggunakan fungsi produksi Cobb-Douglas guna menangkap efek penurunan efisiensi marjinal (**diminishing returns**). Model ini diselesaikan menggunakan pemrograman linier integer campuran (MILP) berbasis diskretisasi serta algoritma genetika (metaheuristik).

Laporan ini menyajikan perumusan model matematika lengkap, penjelasan data, asumsi, justifikasi variabel, serta metode penyelesaian untuk kedua skenario di atas secara terstruktur.

2 Baseline Model (Skenario 1)

Model baseline ini merumuskan masalah **Resource-Constrained Time-Cost Trade-off Problem** (RC-TCTP) dinamis pada proyek dengan durasi diskrit dan biaya pemotongan durasi (**crashing cost**) yang linier.

2.1 Deskripsi Data yang Dimiliki

Model baseline ini dibangun menggunakan tiga berkas data utama yang saling berkaitan:

1. **Data Aktivitas (activity_data_v3.json)**: Menentukan daftar aktivitas proyek, durasi normal (nt_a), durasi minimum setelah crashing maksimum (mt_a), biaya pemangkasan harian (cc_a), hubungan ketergantungan antar-aktivitas (precedence), serta status pengerjaan tugas saat ini.
2. **Kapasitas Sumber Daya (resource_capacity_v3.json)**: Menentukan batas kapasitas harian maksimum (cap_r) untuk setiap jenis sumber daya r .
3. **Kebutuhan Sumber Daya (resource_requirements_v3.json)**: Menentukan kebutuhan harian aktivitas a terhadap sumber daya r ($dem_{a,r}$) ketika aktivitas tersebut sedang aktif berjalan.

2.2 Asumsi dan Limitasi Model

Formulasi model baseline didasarkan pada beberapa asumsi dan limitasi berikut:

- **Linieritas Biaya Crashing**: Pengurangan durasi suatu aktivitas diasumsikan memiliki biaya tambahan konstan per hari. Total biaya crashing untuk suatu tugas adalah hasil kali dari jumlah hari pemotongan dengan biaya harian.
- **Precedence Finish-to-Start**: Hubungan ketergantungan antar-aktivitas yang didukung hanya tipe **Finish-to-Start** (FS) tanpa adanya **lag/lead** waktu.
- **Non-Preemptive**: Aktivitas yang sedang dikerjakan tidak dapat diinterupsi atau diberhentikan sementara hingga benar-benar selesai.
- **Sunk Cost**: Biaya crashing yang terjadi pada masa lalu (aktivitas yang sudah selesai sebelum hari peninjauan saat ini, t_{now}) dianggap sebagai biaya hangus (**sunk cost**) dan diabaikan dari fungsi tujuan optimisasi aktif.

2.3 Notasi

Variabel	Satuan	Deskripsi
Himpunan		
A	-	Himpunan semua aktivitas (tasks) proyek
R	-	Himpunan jenis sumber daya (resources)
E	-	Ketergantungan antar-aktivitas FS, di-indeks oleh (p, a)
A_{active}	-	Aktivitas aktif/belum selesai pada hari peninjauan saat ini (t_{now})
Parameter		
nt_a	hari	Durasi normal aktivitas a
mt_a	hari	Durasi minimum aktivitas a

Variabel	Satuan	Deskripsi
$dem_{a,r}$	satuan/hari	Keperluan harian sumber daya r untuk aktivitas a
cap_r	satuan/hari	Kapasitas harian maksimum sumber daya r
cc_a	\$/hari	Biaya harian untuk crashing aktivitas a
$T_{\max}$	hari	Batas waktu akhir proyek (deadline)
t_{now}	hari	Hari saat ini (hari dijalankannya optimisasi dinamis)
H	hari	Horizon waktu perencanaan proyek (batas atas waktu penyelesaian terjauh)
Variabel Keputusan		
s_a	hari	Hari mulai aktivitas a
d_a	hari	Durasi aktual aktivitas a setelah crashing ($d_a \in [mt_a, nt_a]$)
e_a	hari	Hari selesai aktivitas a ($e_a = s_a + d_a$)
c_a	hari	Jumlah hari pemangkasan durasi (crash days) aktivitas a ($c_a \in [0, nt_a - mt_a]$)
$C_{\max}$	hari	Waktu penyelesaian proyek total (makespan)

2.4 Formulasi Batasan (Constraints)

2.4.1 Batasan Durasi dan Hubungan Crashing

Setiap aktivitas harus memenuhi hubungan fisik antara durasi normal, durasi aktual pengerjaan, dan jumlah hari crashing yang dilakukan:

$$\begin{aligned}
d_a + c_a &= nt_a, \\
e_a &= s_a + d_a, \\
mt_a &\leq d_a \leq nt_a, \\
0 \leq c_a &\leq nt_a - mt_a \quad \forall a \in A.
\end{aligned}$$

Justifikasi: Jika durasi aktivitas dipangkas sebanyak c_a hari, maka durasi pengerjaan aktualnya (d_a) berkurang secara linier namun tidak boleh lebih kecil dari batas teknis durasi minimum (mt_a).

2.4.2 Batasan Ketergantungan (Precedence)

Untuk setiap relasi ketergantungan Finish-to-Start $(p, a) \in E$, aktivitas penerus a hanya boleh dimulai setelah aktivitas pendahulunya p selesai:

$$s_a \geq e_p \quad \forall (p, a) \in E.$$

Justifikasi: Ini menjamin logika urutan pekerjaan jaringan proyek tetap terjaga.

2.4.3 Batasan Kapasitas Sumber Daya Kumulatif

Jumlah total penggunaan suatu sumber daya r oleh seluruh aktivitas yang sedang berjalan pada hari yang sama tidak boleh melebihi kapasitas harian yang tersedia:

$$\sum_{a \in A} dem_{a,r} \cdot \mathbb{1}\{s_a \leq t < e_a\} \leq cap_r, \quad \forall r \in R, \forall t \in \mathcal{T}.$$

Dalam implementasi OR-Tools CP-SAT, batasan ini dinyatakan secara efisien melalui fungsi global:

$$\text{Cumulative}(\{iv_a \mid a \in A \text{ dan } dem_{a,r} > 0\}, \{dem_{a,r}\}, cap_r) \quad \forall r \in R.$$

Di mana iv_a merepresentasikan variabel interval yang ditentukan oleh (s_a, d_a, e_a) . **Justifikasi:** Batasan ini mencegah alokasi sumber daya berlebih ketika beberapa aktivitas yang membutuhkan sumber daya yang sama dijalankan secara paralel.

2.4.4 Batasan Penguncian Status Dinamis (Dynamic State Locking)

Model ini mendukung optimisasi ulang dinamis pada hari t_{now} berdasarkan kondisi riil pelaksanaan proyek di lapangan dengan mengunci status aktivitas:

1. **Aktivitas Belum Mulai** ($status_a = \{\text{not_started}\}$):

$$s_a \geq t_{\text{now}}$$

Justifikasi: Aktivitas hanya dapat dijadwalkan pada hari peninjauan saat ini atau setelahnya.

2. **Aktivitas Sedang Berjalan** ($status_a = \{\text{in_progress}\}$):

$$s_a = \text{actual_start}_a,$$

$$e_a \geq t_{\text{now}} + 1,$$

$$d_a \geq \max(mt_a, \text{elapsed}_a + 1)$$

Di mana $\text{elapsed}_a = t_{\text{now}} - \text{actual_start}_a$. **Justifikasi:** Hari mulai dikunci sesuai realisasi aktual di lapangan. Aktivitas tidak boleh selesai secara retrospektif sebelum hari ini ($e_a \geq t_{\text{now}} + 1$) dan durasi tidak dapat di-crash di bawah hari-hari yang telah terlewati (elapsed_a).

3. **Aktivitas Sudah Selesai** ($status_a = \{\text{completed}\}$):

$$s_a = \text{actual_start}_a,$$

$$e_a \leq t_{\text{now}},$$

$$d_a = \text{actual_duration}_a$$

Justifikasi: Mengunci seluruh variabel aktivitas agar sesuai dengan realisasi sejarah proyek dan tidak diubah oleh solver selama proses optimisasi ulang.

2.4.5 Batasan Akhir Proyek (Makespan)

Makespan proyek (C_{max}) dibatasi oleh waktu selesai aktivitas paling setelah:

$$C_{\text{max}} \geq e_a \quad \forall a \in A.$$

2.5 Formulasi Objektif

2.5.1 Mode A: Biaya Minimum dengan Batas Waktu (Cost-Driven)

Digunakan jika target batas waktu proyek ($T_{\max}$) ditentukan. Tujuannya adalah meminimalkan total biaya crashing tambahan untuk aktivitas yang masih aktif:

$$\min \sum_{a \in A_{\text{active}}} cc_a \cdot c_a$$

With batasan tambahan:

$$C_{\max} \leq T_{\max}.$$

2.5.2 Mode B: Makespan Minimum (Time-Driven)

Tujuan utamanya adalah mempercepat penyelesaian proyek secepat mungkin tanpa memperhitungkan biaya crashing:

$$\min C_{\max}.$$

2.6 Metode Penyelesaian

Model baseline diselesaikan menggunakan **Google OR-Tools CP-SAT Solver**. CP-SAT dipilih karena menyediakan variabel interval (**IntervalVar**) secara native yang mempermudah representasi durasi tugas dan memiliki algoritma propagasi batasan kumulatif (**AddCumulative**) yang sangat efisien untuk memecahkan masalah penjadwalan dengan batasan sumber daya berkapasitas terbatas.

Integer Scaling: Karena solver CP-SAT hanya menerima nilai koefisien berupa bilangan bulat (integer), parameter biaya crashing riil (cc_a) yang berbentuk desimal dikalikan dengan faktor skala $S = 10^{\{\text{max_dp}\}}$ (di mana max_dp adalah presisi desimal maksimum dari data biaya). Koefisien tujuan ter-skala dihitung dengan $\text{coeff}_a = \lfloor cc_a \cdot S \rfloor$. Setelah diperoleh solusi optimal, total biaya riil diperoleh dengan membagi kembali nilai objektif solver dengan S .

3 Novel Model (Skenario 2)

Di sini, kami memformulasikan model TCTP baru. Secara khusus, dibuat model RC-WTCTP berbasis **workforce** (bukan durasi seperti CTCP atau mode seperti DTCTP) dengan fungsi biaya **endogenous** (tidak **exogenous** seperti pada TCTP umumnya) yang diturunkan dari model utilitas Cobb-Douglas.

3.1 Deskripsi Data yang Dimiliki

Model ini menggunakan berkas data di direktori `data/original-data` yang diimpor dari perangkat lunak manajemen proyek (seperti MS Project):

1. **Data Aktivitas (`task_table.json`)**: Berisi daftar seluruh aktivitas (V), level outline, durasi baseline, tanggal mulai dan selesai, serta dependensi beserta nilai **lag/lead**.
2. **Data Sumber Daya (`resource_table.json`)**: Berisi daftar jenis tenaga kerja (K), kapasitas maksimal harian (C_k dalam persen atau unit), dan tarif upah standar per jam (r_k).
3. **Data Alokasi Tugas (`assignment_table.json`)**: Menghubungkan aktivitas dengan tenaga kerja yang ditugaskan (K_i), mencakup usaha kerja baseline ($W_{i,k}$ dalam jam) dan persentase alokasi harian baseline ($U_{i,k}$).

3.2 Asumsi dan Limitasi Model

Formulasi model Cobb-Douglas didasarkan pada beberapa asumsi realistis berikut:

- **Diminishing Returns**: Menambah jumlah pekerja pada suatu aktivitas (**overcrowding**) atau memperpanjang jam kerja (**overtime**) akan mempercepat pengerjaan aktivitas, namun dengan efisiensi marjinal yang menurun. Hal ini dimodelkan dengan eksponen elastisitas $\alpha, \beta \in (0, 1)$ pada fungsi Cobb-Douglas.
- **Koordinasi dan Kelelahan**: Overcrowding menyebabkan hilangnya efisiensi karena peningkatan beban koordinasi antar-pekerja. Overtime menurunkan produktivitas pekerja karena kelelahan (**fatigue**).
- **Tarif Lembur Lebih Tinggi**: Jam kerja lembur (

τ

) dikenakan tarif upah yang lebih tinggi daripada jam reguler (dikali multiplier upah lembur $r'_k = \text{ot_mult} \cdot r_k$).

- **Non-Preemptive**: Pengerjaan tugas bersifat kontinu dari hari mulai hingga selesai (non-preemptive).

3.3 Notasi

Variabel	Satuan	Deskripsi
Himpunan		
V	-	Himpunan semua aktivitas, termasuk awal (0) dan akhir (n) semu
V_2	-	Himpunan aktivitas outline level 2 (aktivitas riil yang dapat di-crash)

Variabel	Satuan	Deskripsi
K_i	-	Himpunan SDM yang ditugaskan ke aktivitas i
$E_{\{FS\}}, E_{\{SS\}}, E_{\{FF\}}$	-	Himpunan relasi ketergantungan Finish-to-Start, Start-to-Start, dan Finish-to-Finish
$\mathcal{T}$	-	Himpunan hari dalam horizon proyek, $\mathcal{T} = \{1, 2, \dots, s, H\}$
Parameter		
$W_{i,k}$	jam	Usaha kerja baseline SDM k untuk aktivitas i
$U_{i,k}$	-	Alokasi harian baseline SDM k untuk aktivitas i (proporsi dari 8 jam)
C_k	unit	Kapasitas SDM k yang tersedia per hari
r_k	Rp/jam	Tarif reguler SDM k
r'_k	Rp/jam	Tarif lembur SDM k , dengan $r'_k \geq r_k$
α	-	Eksponen Cobb–Douglas untuk overcrowding ($0 < \alpha < 1$)
β	-	Eksponen Cobb–Douglas untuk overtime ($0 < \beta < 1$)
T_d	hari	Target waktu penyelesaian proyek (contractual deadline)
$c_{\{late\}}$	Rp/hari	Penalti keterlambatan
$c_{\{early\}}$	Rp/hari	Bonus penyelesaian awal
$c_{\{ind\}}$	Rp/hari	Biaya overhead proyek tidak langsung harian
δ_{ij}	hari	Lag/lead antara aktivitas i dan j
t_0	hari	Hari saat ini peninjauan proyek
Variabel Keputusan		
$x_{i,k}$	-	Pengali overcrowding SDM k untuk aktivitas $i \in V_2$
$\tau_{i,k}$	jam/hari	Lama overtime harian SDM k untuk aktivitas $i \in V_2$
s_i, f_i	hari	Hari mulai dan selesai aktivitas i
d_i	hari	Durasi aktual aktivitas i ($d_i = f_i - s_i$)
L, E	hari	Hari proyek selesai setelah / sebelum T_d (lateness/earliness)
$y_{i,t}$	-	Variabel biner: bernilai 1 jika aktivitas i aktif pada hari t
$z_{i,t}$	-	Variabel biner: bernilai 1 jika aktivitas i dimulai pada hari t

3.4 Formulasi dan Penurunan Matematis Cobb-Douglas

3.4.1 1. Durasi Aktivitas

Berdasarkan data penjadwalan baseline, durasi normal aktivitas i dapat dihitung dari kebutuhan kerja per tugas:

$$d_i^{\{baseline\}} = \max_{k \in K_i} \frac{W_{i,k}}{8U_{i,k}}.$$

Biaya baseline untuk tenaga kerja k pada tugas i adalah $z_{i,k}^{\{baseline\}} = W_{i,k}r_k$.

Jika kita melakukan crashing secara naif dengan menambahkan pengali tenaga kerja $x_{i,k}$ dan jam lembur $\tau_{i,k}$, durasi baru dihitung sebagai:

$$d'_i = \max_{k \in K_i} \frac{W_{i,k}}{(8 + \tau_{i,k})(x_{i,k}U_{i,k})}.$$

Namun, rumusan naif ini mengakibatkan total biaya labor konstan:

$$z'_{i,k} = d'_i \cdot x_{i,k}U_{i,k} \cdot (8 + \tau_{i,k}) \cdot r_{i,k} = W_{i,k}r_{i,k} = z_{i,k}^{\{\text{baseline}\}}.$$

Hal ini tidak realistis karena seolah-olah penambahan pekerja dan lembur tidak memicu inefisiensi biaya tambahan.

Untuk mengatasi ini, diperkenalkan fungsi produksi Cobb-Douglas dengan efisiensi marginal menurun (**diminishing returns**):

$$d'_i = \max_{k \in K_i} \frac{W_{i,k}}{A_{\{i,k\}}(x_{i,k}U_{i,k})^\alpha(8 + \tau_{i,k})^\beta}.$$

Dengan memilih konstanta kalibrasi $A_{\{i,k\}} = U_{i,k}^{1-\alpha}8^{1-\beta}$ agar durasi baru bernilai sama dengan baseline ketika $x_{i,k} = 1$ and $\tau_{i,k} = 0$, maka durasi ter-crash dapat disederhanakan menjadi:

$$d'_i = \max_{k \in K_i} \frac{W_{i,k}}{8U_{i,k}} \cdot \frac{1}{x_{i,k}^\alpha} \cdot \left(\frac{8}{8 + \tau_{i,k}} \right)^\beta.$$

Di mana $\frac{1}{x_{i,k}^\alpha}$ menyatakan faktor perlambatan akibat inefisiensi koordinasi overcrowding, dan $\left(\frac{8}{8 + \tau_{i,k}} \right)^\beta$ menyatakan faktor perlambatan akibat kelelahan lembur.

3.4.2 2. Penurunan Fungsi Biaya Tenaga Kerja

Jumlah hari pengerjaan tugas adalah d'_i . Total hari-orang untuk SDM k pada tugas i adalah $d'_i \cdot x_{i,k}U_{i,k}$. Biaya per hari-orang mencakup upah reguler (8 jam dengan tarif r_k) dan upah lembur ($\tau_{i,k}$ jam dengan tarif lembur r'_k):

$$\text{Biaya Harian Orang} = 8r_k + \tau_{i,k}r'_k.$$

Dengan mengalikan jumlah hari-orang dengan biaya harian, diperoleh total biaya tenaga kerja $z'_{i,k}$ untuk SDM k pada aktivitas i :

$$z'_{i,k} = d'_i \cdot x_{i,k}U_{i,k} \cdot (8r_k + \tau_{i,k}r'_k)$$

$$z'_{i,k} = W_{i,k} \cdot x_{i,k}^{1-\alpha} \cdot \left(\frac{8}{8 + \tau_{i,k}} \right)^\beta \cdot \left(r_k + \frac{\tau_{i,k}}{8}r'_k \right).$$

Persamaan biaya ini secara elegan memodelkan **diminishing returns**: karena $1 - \alpha < 1$, peningkatan overcrowding ($x > 1$) akan menghasilkan total biaya tenaga kerja yang lebih tinggi daripada baseline untuk menyelesaikan jumlah pekerjaan yang sama.

3.4.3 Contoh Kasus Crashing

Perhatikan ilustrasi tugas di bawah ini:

Submit shop drawings and order long lead items - roofing

Roofing Contractor Management

80h

100%

Secara normal, pengerjaan tugas ini membutuhkan **Roofing Contractor Management** untuk bekerja 8 jam per hari selama 10 hari (total 80 jam kerja baseline).

1. **Mekanisme Overtime:** Jika pekerja diminta untuk lembur $\tau = 1$ jam per hari (total 9 jam kerja per hari), dengan efek kelelahan $\beta = 0.3$, durasi baru menjadi $d' = 10 \cdot \left(\frac{8}{9}\right)^{0.3} \approx 9.6$ hari (dibulatkan menjadi 10 hari atau dipercepat 1 hari tergantung grid). Upah jam lembur dihitung dengan tarif r'_k lebih tinggi.
2. **Mekanisme Overcrowding:** Jika kita menambah satu pekerja lagi ($x = 2.0$) dengan efek koordinasi $\alpha = 0.6$, durasi baru menjadi $d' = 10 \cdot \frac{1}{2^{0.6}} \approx 6.6$ hari. Jumlah hari-pekerja meningkat menjadi $6.6 \cdot 2 = 13.2$ hari-orang, yang berarti biaya labor total naik sebesar $2^{1-0.6} = 2^{0.4} \approx 1.32$ kali biaya baseline.

3.5 Formulasi Batasan Lengkap (Novel Model)

3.5.1 1. Batasan Durasi (Cobb-Douglas)

Untuk setiap aktivitas $i \in V_2$ dan tenaga kerja $k \in K_i$:

$$f_i - s_i \geq \frac{W_{i,k}}{8U_{i,k}} \cdot \frac{1}{x_{i,k}^\alpha} \cdot \left(\frac{8}{8 + \tau_{i,k}} \right)^\beta.$$

Untuk aktivitas yang tidak dapat di-crash ($i \in V - V_2$), durasinya mengikuti baseline:

$$f_i - s_i \geq d_i^{\{\text{baseline}\}}.$$

3.5.2 2. Batasan Ketergantungan (Precedence) dengan Lag

Mendukung tiga tipe relasi ketergantungan utama:

$$s_j \geq f_i + \delta_{ij} \quad \forall (i, j) \in E_{\{\text{FS}\}},$$

$$s_j \geq s_i + \delta_{ij} \quad \forall (i, j) \in E_{\{\text{SS}\}},$$

$$f_j \geq f_i + \delta_{ij} \quad \forall (i, j) \in E_{\{\text{FF}\}}.$$

3.5.3 3. Batasan Waktu Sekarang (Anchoring)

Aktivitas yang belum selesai tidak boleh dijadwalkan sebelum hari saat ini t_0 :

$$s_i \geq t_0 \quad \forall i \in V \setminus \overline{\{V\}}.$$

Sedangkan untuk aktivitas yang sedang berjalan ($i \in V^{\{\text{aktif}\}}$), sisa jam kerja $W_{i,k}$ disesuaikan menjadi $W_{i,k}^{\{\text{sisal}\}} = W_{i,k}(1 - p_{\{i,k\}})$ dan dipaksa mulai tepat pada $s_i = t_0$.

3.5.4 4. Batasan Aktivasi Harian (Time-Indexed) dan Kontinuitas

Mendefinisikan biner aktivasi $y_{i,t} = 1$ jika aktivitas i sedang dikerjakan pada hari t , dan biner mulai tugas $z_{i,t} = 1$ jika tugas i dimulai pada hari t :

$$\begin{aligned} s_i &= \sum_{t \in \mathcal{T}} t \cdot z_{i,t}, \\ d_i &= \sum_{t \in \mathcal{T}} y_{i,t}, \\ f_i &= s_i + d_i, \\ z_{i,t} &\geq y_{i,t} - y_{i,t-1} \quad \forall t > ES_i, \\ \sum_{t \in \mathcal{T}} z_{i,t} &\leq 1. \end{aligned}$$

Justifikasi: Batasan kontiguitas ini memastikan tugas berjalan terus-menerus tanpa terputus (non-preemptive).

3.5.5 5. Batasan Kapasitas Harian dan Linearisasi McCormick

Penggunaan harian kapasitas reguler ($u_{i,k,t}$) dan lembur ($o_{i,k,t}$) untuk setiap tenaga kerja k pada hari t tidak boleh melebihi kapasitas maksimumnya:

$$\begin{aligned} \sum_{i \in V} u_{i,k,t} &\leq C_k \cdot 8.0 \\ \sum_{i \in V} o_{i,k,t} &\leq C_k \cdot \tau_{\max,k} \end{aligned}$$

Di mana alokasi harian per aktivitas bernilai $u_{i,k,t} = y_{i,t} \cdot (U_{i,k} \cdot 8.0 \cdot x_{i,k})$ dan $o_{i,k,t} = y_{i,t} \cdot (U_{i,k} \cdot \tau_{i,k})$.

Karena terdapat perkalian variabel kontinu ($x_{i,k}$ dan $\tau_{i,k}$) dengan variabel biner ($y_{i,t}$), diterapkan linearisasi McCormick Envelope:

$$\begin{aligned} u_{i,k,t} &\leq u_{\max} y_{i,t} \\ u_{i,k,t} &\leq \text{u_task_day}_{i,k} \\ u_{i,k,t} &\geq \text{u_task_day}_{i,k} - u_{\max}(1 - y_{i,t}) \end{aligned}$$

Di mana $\text{u_task_day}_{i,k} = U_{i,k} \cdot 8.0 \cdot x_{i,k}$ dan $u_{\max} = U_{i,k} \cdot 8.0 \cdot x_{\max,k}$. (Penanganan yang sama dilakukan untuk variabel overtime $o_{i,k,t}$). **Justifikasi:** Linearisasi ini menghilangkan sifat bilinearitas model sehingga solver MILP standar dapat menyelesaikan model secara optimal global.

3.6 Formulasi Objektif Berdasarkan Skenario

3.6.1 Skenario A: Optimisasi Waktu-Biaya (Time-Cost Tradeoff)

Meminimalkan total pengeluaran proyek yang mencakup biaya tenaga kerja, biaya tidak langsung (**overhead**), penalti keterlambatan, dan bonus penyelesaian awal:

$$\min Z = \sum_{\{i \in V\}} \sum_{\{k \in K_i\}} z'_{\{i,k\}} + c_{\{\text{ind}\}} T_{\{\text{proj}\}} + c_{\{\text{late}\}} L - c_{\{\text{early}\}} E.$$

Di mana keterlambatan (L) dan penyelesaian awal (E) dilinearkan sebagai:

$$T_{\{\text{proj}\}} - T_d = L - E, \quad L, E \geq 0.$$

3.6.2 Skenario B: Batasan Anggaran (Budget-Constrained)

Meminimalkan durasi penyelesaian proyek ($T_{\{\text{proj}\}}$) dengan menjaga agar total biaya pengeluaran tidak melebihi anggaran maksimal ($B_{\{\text{max}\}}$):

$$\begin{aligned} & \min T_{\{\text{proj}\}} \\ \text{s.t.} \quad & Z \leq B_{\{\text{max}\}}. \end{aligned}$$

3.6.3 Skenario C: Batasan Batas Waktu (Deadline-Constrained)

Meminimalkan total biaya pengeluaran (Z) dengan batasan keras bahwa waktu penyelesaian proyek tidak boleh melewati deadline T_d :

$$\begin{aligned} & \min Z \\ \text{s.t.} \quad & T_{\{\text{proj}\}} \leq T_d. \end{aligned}$$

4 Metode Penyelesaian Skenario 2

Model Cobb-Douglas asli merupakan model **MINLP (Mixed-Integer Non-Linear Programming)** yang non-konveks karena persamaan durasi dan biaya mengandung eksponen pecahan. Untuk menyelesaikan model ini secara efisien dan andal, digunakan dua pendekatan:

4.1 1. MILP berbasis Diskretisasi Grid (Direkomendasikan)

Metode ini mendiskretkan ruang pencarian kontinu pengali overcrowding $x_{i,k}$ dan overtime harian $\tau_{i,k}$ menjadi beberapa titik grid tertentu:

- $x_g \in \{1.0, 1.25, 1.5, 1.75, 2.0\}$
- $\tau_g \in \{0.0, 1.0, 2.0, 3.0, 4.0\}$

Definisikan variabel keputusan biner baru $\xi_{i,k}^{m,n} \in \{0,1\}$ yang bernilai 1 jika SDM k pada tugas i memilih titik grid overcrowding ke- m dan overtime ke- n . Dengan diskretisasi ini, nilai durasi $d_{\{i,m,n\}}$ dan biaya harian labor cost $cost_{i,m,n}$ untuk masing-masing kombinasi grid dihitung terlebih dahulu sebelum optimisasi (**precomputed**). Seluruh konstrain durasi dan biaya menjadi fungsi linier terhadap variabel biner $\xi_{i,k}^{m,n}$:

$$d_i = \sum_m \sum_n \xi_{i,k}^{m,n} \cdot d_{\{i,m,n\}}.$$

Model ini kemudian dimodelkan menggunakan **Pyomo** dan diselesaikan menggunakan solver MILP komersial/open-source seperti **CBC** atau **HiGHS** hingga mencapai jaminan solusi optimal global dalam hitungan detik.

4.2 2. Pendekatan Metaheuristik (Genetic Algorithm)

Sebagai alternatif pembanding untuk ruang pencarian kontinu tanpa diskretisasi, diimplementasikan algoritma genetika (GA) menggunakan pustaka **pymoo** di Python:

- **Representasi Kromosom:** Variabel keputusan (x, τ) dikodekan langsung sebagai vektor bilangan real.
- **Fungsi Penalti:** Karena GA sulit menangani batasan secara langsung, kendala precedence dan kapasitas sumber daya ditambahkan sebagai penalti kuadratis ke dalam fungsi objektif jika terjadi pelanggaran (**precedence violation penalty**).
- **Operator Genetika:** Menggunakan operator seleksi turnamen, persilangan SBX (*Simulated Binary Crossover*), dan mutasi PM (*Polynomial Mutation*).

Meskipun metode GA dapat menangani fungsi Cobb-Douglas asli tanpa diskretisasi, ia tidak memberikan jaminan optimalitas global dan membutuhkan waktu komputasi yang lebih lama untuk konvergensi dibandingkan dengan pendekatan MILP diskret.

5 Kesimpulan dan Perbandingan Model

Karakteristik	Model Baseline (CP-SAT)	Model Novel (Cobb-Douglas)
Input Biaya	Eksplisit diketahui per hari crashing per tugas (\$/hari).	Ditentukan secara endogen dari tarif reguler/lembur SDM (\$/jam).
Tuas Akselerasi	Langsung memotong hari durasi tugas.	Menambah pekerja (overcrowding) & menambah jam kerja lembur (overtime).
Sifat Efisiensi	Efisiensi konstan (biaya linier terhadap waktu pemangkasan).	Efisiensi menurun (diminishing returns) akibat koordinasi & kelelahan.
Tipe Precedence	Finish-to-Start (FS) sederhana tanpa lag.	Finish-to-Start (FS), Start-to-Start (SS), Finish-to-Finish (FF) dengan lag.
Metode Solver	Constraint Programming (OR-Tools CP-SAT).	MILP (Pyomo + CBC) via Diskretisasi Grid & Metaheuristik (pymoo GA).

Secara ringkas, jika manajer proyek memiliki estimasi biaya crashing langsung dan mengabaikan efek kelelahan kerja, **Model Baseline** adalah pilihan tercepat. Namun, untuk alokasi proyek yang lebih realistis dan taktis yang memperhatikan kelelahan pekerja serta batasan kapasitas harian terperinci, **Model Novel Cobb-Douglas** memberikan solusi yang jauh lebih akurat dan dapat memodelkan trade-off operasional yang sebenarnya di lapangan.